

## Question 1: Complete Banking System

Design a complete banking system using OOP.

Create classes such as:

- Bank
- Customer
- Account
- SavingsAccount
- CurrentAccount
- Transaction
- Loan
- Employee

Requirements:

- A customer can have multiple accounts.
- Support deposits and withdrawals.
- Support transfers between accounts.
- Maintain transaction history.
- Implement different account types.
- Implement interest calculation.
- Support loans.
- Maintain account balances securely.
- Prevent invalid transactions.

Create custom exceptions such as:

- InsufficientBalanceError
- InvalidAmountError
- AccountNotFoundError

Use abstraction and polymorphism wherever appropriate.

## Question 2: Airline Reservation System

Build an airline reservation system.

Create classes such as:

- Airline
- Airport
- Flight
- Aircraft
- Passenger
- Seat
- Booking
- Ticket

- **Payment**

Requirements:

- Flights have departure and arrival airports.
- Aircraft contain multiple seats.
- Seats can have different classes.
- Passengers can book seats.
- Prevent double booking.
- Support cancellation.
- Calculate ticket prices based on seat class.
- Maintain booking history.
- Generate a unique ticket number.

Implement different seat types using inheritance.

## Question 3: Ride-Sharing Application

Build a simplified application similar to Uber/Ola.

Create:

- **User**
- **Rider**
- **Driver**
- **Vehicle**
- **Ride**
- **Location**
- **Payment**
- **Rating**

Requirements:

- Riders can request rides.
- Drivers can accept rides.
- Match riders with available drivers.
- Calculate distance between locations.
- Calculate fare based on distance and vehicle type.
- Track ride status.
- Support ride cancellation.
- Allow riders to rate drivers.
- Allow drivers to rate riders.

Implement appropriate state transitions:

- **REQUESTED**
- **ACCEPTED**
- **DRIVER\_ARRIVING**
- **IN\_PROGRESS**
- **COMPLETED**

- CANCELLED

## Question 4: E-Commerce Backend

Design a complete object-oriented e-commerce backend.

Implement:

- User
- Customer
- Admin
- Product
- Category
- Inventory
- Cart
- Order
- OrderItem
- Payment
- Coupon
- Shipping

Requirements:

- Product inventory management.
- Shopping cart management.
- Order creation.
- Discount coupons.
- Multiple payment methods.
- Shipping calculation.
- Order cancellation.
- Product returns.
- Inventory updates after purchase.
- Order history.

Use abstraction and polymorphism for:

- Payments
- Discounts
- Shipping strategies

## Question 5: Hotel Reservation System

Build a hotel booking system.

Create:

- Hotel
- Room
- Customer
- Reservation

- Payment
- Invoice

Room types should include:

- SingleRoom
- DoubleRoom
- DeluxeRoom
- Suite

Requirements:

- Search available rooms.
- Book rooms for specific dates.
- Prevent overlapping reservations.
- Calculate total price.
- Apply seasonal pricing.
- Support cancellation.
- Generate invoices.
- Track customer booking history.

Use classes and appropriate design patterns where necessary.

## Question 6: Parking Lot Management System

Design a multi-level parking lot system.

Create:

- ParkingLot
- ParkingFloor
- ParkingSpot
- Vehicle
- Car
- Bike
- Truck
- ParkingTicket
- Payment

Requirements:

- Multiple parking floors.
- Different parking spot sizes.
- Assign the appropriate spot based on vehicle type.
- Generate parking tickets.
- Calculate parking fees based on duration.
- Release spots when vehicles leave.
- Display available spots.
- Prevent parking when the lot is full.

Support different pricing strategies using polymorphism.

## Question 7: ATM Machine

Design an ATM system using OOP.

Create classes such as:

- `ATM`
- `Bank`
- `Account`
- `Card`
- `Customer`
- `Transaction`
- `CashDispenser`

Support:

- Card insertion.
- PIN authentication.
- Balance inquiry.
- Cash withdrawal.
- Cash deposit.
- Fund transfer.
- PIN change.
- Transaction history.

Implement appropriate custom exceptions.

Example:

- `InvalidPINError`
- `InsufficientBalanceError`
- `InsufficientCashError`
- `CardBlockedError`
- `InvalidTransactionError`

The ATM should also maintain its available cash denominations.

For example:

- ₹500
- ₹200
- ₹100
- ₹50

The withdrawal system should determine an appropriate combination of notes.

## Question 8: Online Learning Platform

Build an object-oriented online learning platform similar to Udemy.

Create:

- User
- Student
- Instructor
- Course
- Module
- Lesson
- VideoLesson
- QuizLesson
- AssignmentLesson
- Enrollment
- Certificate
- Payment

Requirements:

- Instructors can create courses.
- Courses contain multiple modules.
- Modules contain multiple lessons.
- Students can enroll in courses.
- Track student progress.
- Support quizzes.
- Calculate quiz scores.
- Mark lessons as completed.
- Generate certificates after course completion.
- Support different payment methods.

Use inheritance and polymorphism for different lesson types.

## Question 9: Stock Trading Platform

Design a simplified stock trading system.

Create classes:

- User
- Account
- Stock
- Portfolio
- Order
- BuyOrder
- SellOrder
- Trade
- Market
- Broker

Requirements:

- Users can maintain portfolios.

- Users can buy and sell stocks.
- Maintain available cash.
- Maintain stock quantities.
- Create buy and sell orders.
- Validate orders.
- Execute trades.
- Maintain transaction history.
- Calculate portfolio value.
- Calculate profit/loss.

Support different order types:

- MARKET\_ORDER
- LIMIT\_ORDER
- STOP\_ORDER

Use polymorphism to implement different order execution strategies.

## Question 10: Design a Mini Social Media Platform

Build a simplified social media platform similar to Twitter/X.

Create classes such as:

- User
- Profile
- Post
- Comment
- Like
- Follow
- Message
- Notification
- Feed

Requirements:

- Users can create profiles.
- Users can follow other users.
- Users can create posts.
- Users can like posts.
- Users can comment on posts.
- Users can delete their posts.
- Users can send direct messages.
- Generate notifications.
- Generate a personalized feed.
- Support pagination for posts.
- Maintain follower/following relationships.
- Prevent users from performing unauthorized operations.

Use:

- Abstract base classes where appropriate.
- Inheritance.
- Polymorphism.
- Encapsulation.
- Composition.
- Custom exceptions.
- Properties using `@property`.
- Class methods.
- Static methods.
- Magic methods such as **`str`**, **`repr`**, **`eq`**, and **`len`**.
- Type hints.
- Dataclasses where appropriate.

Implement the system so that individual components are loosely coupled and can be replaced without modifying the entire application.